

透明・ハッシュベース汎用ゼロ知識証明の設計論

ZK-STARK、Ligero、Aurora の数理・実装構造・応用・比較

有限体 F_{17} 上の toy relation $y=(x+1)(x+2)(x+3)$ を用いた統一的チュートリアル

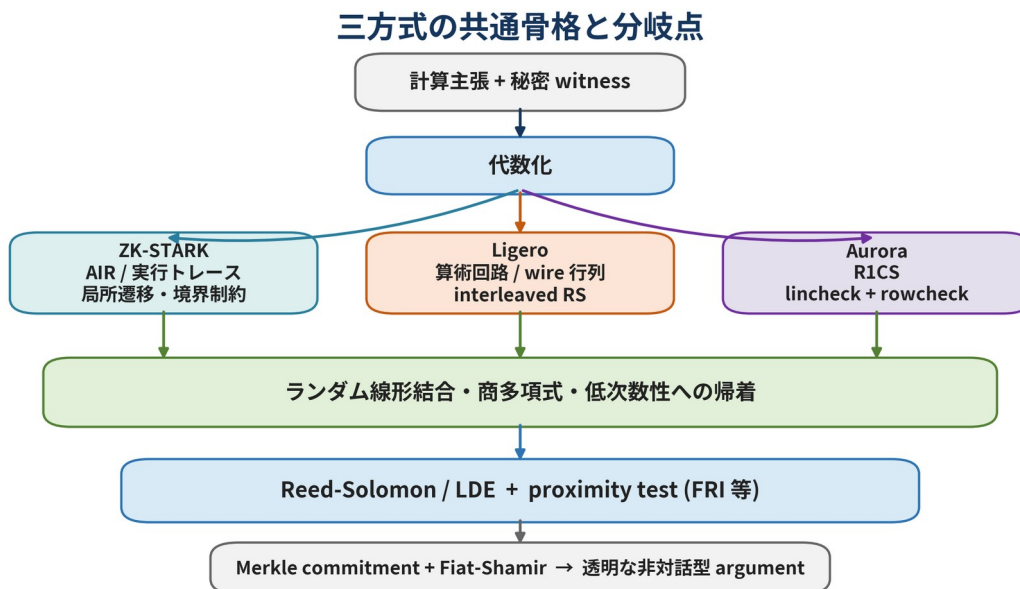


図 1 三方式の共通骨格と分岐点

対象文献

- Ben-Sasson et al., “Scalable, transparent, and post-quantum secure computational integrity”, 2018
Ames et al., “Ligero: Lightweight Sublinear Arguments Without a Trusted Setup”, extended version, 2022
Ben-Sasson et al., “Aurora: Transparent Succinct Arguments for R1CS”, 2019

2026 年 7 月

要旨

本稿は、透明なセットアップとハッシュベースの非対話化を共有する三つの汎用ゼロ知識証明方式、ZK-STARK、Ligero、Aurora を、原論文の順序に従って体系的に解説するものである。完全性・健全性・零知識性といった ZKP 一般の定義は既知とし、計算の代数化、Reed-Solomon 符号、低次数拡張、Interactive Oracle Proof、低次数近接検査、零知識マスキング、Merkle commitment および Fiat-Shamir 変換に焦点を置く。

三方式の差は、同じ暗号学的外殻を使いながら、何を「局所的な代数関係」として符号化するかにある。ZK-STARK は実行トレースと AIR を採用し、長い逐次計算や簡潔に記述されたプログラムに適する。Ligero は拡張 witness をほぼ平方行列に並べ、interleaved Reed-Solomon 符号と三種類の検査によって平方根通信量を得る。Aurora は R1CS を lincheck と rowcheck に分解し、単変数 sumcheck を介して対数照会数を達成する。

さらに、有限体 F_{17} 上の公開出力 $y=9$ と秘密 witness $x=2$ に対する関係 $y=(x+1)(x+2)(x+3)$ を共通例として用い、各方式で証明者がどの表現を構成し、検証者がどのランダム線形結合と低次数性を検査するかを具体的な数値で追跡する。 F_{17} は暗号学的安全性を持たないため、ここでの例はプロトコル構造を可視化する教育用モデルであり、実用パラメータの代替ではない。

キーワード: ZK-STARK、Ligero、Aurora、AIR、R1CS、Reed-Solomon 符号、FRI、IOP、sumcheck、透明セットアップ

目次

1. 本稿の範囲と比較方法
2. 三方式に共通する技術基盤
3. ZK-STARK: 実行トレースと低次数性
4. Ligero: interleaved RS と平方根通信量
5. Aurora: R1CS、lincheck、単変数 sumcheck
6. 三方式の横断比較
7. 応用設計と方式選択
8. 結論
- 付録 A. F_{17} toy computation の完全データ
- 付録 B. エビデンス表と比較上の注意
- 参考文献

1. 本稿の範囲と比較方法

本稿は三方式を「透明・ハッシュベース・IOP 系」という共通クラスに置き、その内部設計を比較する。ZKP 一般の導入、標準的な安全性定義、NP 完全性の説明は省略する。一方、方式差を生む代数化モデル、符号化、近接検査、ランダム化、計算量については詳細に扱う。

比較では、漸近量、同一論文内の制御された実験、異なる論文間の歴史的測定値を明確に分離する。異なる方式の実装値は、有限体、security parameter、対象関係、回路トポロジー、ハードウェア、並列度、Merkle 木の構成、Fiat-Shamir の反復数によって大きく変化するためである。したがって、単一の「最速方式」を断定するのではなく、どの設計点で優位になるかを記述する。

比較の原則

表中の $O(\cdot)$ は原論文が同じ計算モデルに写像した場合の漸近比較である。実測値は必ず出典、security level、field、workload を併記する。2019 年 Aurora 論文の比較と 2022 年 Ligerio 改訂版の比較が異なる順位を示すことは矛盾ではなく、実装・解析・パラメータの更新が定数項を大きく変えることの例である。

表 1 本稿で頻出する記号

記号	意味
F	有限体。toy 例では $F=F_{17}$ である。
H	計算値を配置する trace/message domain。
L	低次数拡張を評価する oracle domain。
$Z_H(X)$	H 上で消える消失多項式 $\prod_{a \in H} (X-a)$ 。
$RS[L,d]$	L 上の次数 d 未満の多項式評価からなる Reed-Solomon 符号。
N	回路・R1CS の規模を表す代表量。
λ	security parameter。

2. 三方式に共通する技術基盤

2.1 計算を「多項式が低次数である」という主張へ変換する

三方式は、計算結果の正しさをそのまま検査するのではなく、計算履歴または制約充足ベクトルを有限体上の関数として配置し、その補間多項式が所定の次数境界と代数恒等式を満たすことへ帰着する。計算の局所制約が全点で成立するなら、制約多項式は対応する domain の消失多項式で割り切れる。したがって、商多項式の存在と低次数性を検査すれば、多数の制約を少数のランダム照会へ圧縮できる。

$$C(X)=Z_H(X) \cdot Q(X) \Leftrightarrow C(a)=0 \text{ for every } a \in H$$

ZK-STARK では C が隣接 trace 行の遷移制約から得られる。Ligero では行列化された wire 値に対する線形制約と二次制約が C に相当する。Aurora では R1CS の三つの線形像 Az 、 Bz 、 Cz の整合性、および Hadamard 積 $Az \circ Bz = Cz$ が C に相当する。

2.2 Reed-Solomon 符号と低次数拡張

有限集合 H 上の値ベクトル v は、次数 $|H|-1$ 未満の補間多項式 $\hat{v}$ によって一意に表現できる。より大きい domain L 上に $\hat{v}$ を評価したものが low-degree extension であり、同時に Reed-Solomon codeword である。LDE には二つの役割がある。第一に、元の値を冗長化して局所照会から大域的整合性を検査可能にする。第二に、 H 外の評価値をランダム化する余地を作り、零知識性を実現する。

$$\text{LDE}_L(v) := (\hat{v}(\alpha))_{\alpha \in L}, \quad \deg(\hat{v}) < |H|$$

実装上の主要コストは補間と多点評価であり、FFT-friendly field と加法 FFT または乗法 FFT が重要になる。三方式の prover complexity に現れる $\log N$ 因子の多くは、この符号化と多項式演算に由来する。

2.3 IOP、proximity test、Merkle commitment

IOP では証明者が長い oracle を送るが、検証者は oracle 全体を読む必要がない。検証者は少数の位置だけを照会し、oracle が Reed-Solomon 符号またはその近傍にあることを proximity test で検査する。情報理論的 IOP を現実の argument に変換する際には、各 oracle を Merkle 木へ commit し、ランダム照会点の値と authentication path だけを開示する。

非対話化では、検証者の public coin を Fiat-Shamir で transcript hash から導出する。透明性とは、秘密の trapdoor を生成する setup ceremony を必要としないという意味であり、hash function、field、domain、security parameter 等の公開パラメータは依然として必要である。

安全性の層

最終 argument の健全性は、(i) 代数化の健全性、(ii) ランダム線形結合の失敗確率、(iii) low-degree/proximity test の誤り、(iv) Merkle binding、(v) Fiat-Shamir をランダムオラクルとして扱う仮定、の合成である。これらを一つの「soundness error」に潰さず、層別に監査する必要がある。

2.4 三方式に共通する零知識化のパターン

三方式は、異なる記法を用いるが、零知識化の基本操作は共通している。秘密値を一意な低次数多項式で符号化すると、少数照会であっても秘密の線形情報が漏れる可能性がある。そこで、秘密値を固定する domain H 上ではゼロとなる多項式 $Z_H(X)$ のランダム倍を加える。

$$\tilde{f}(X) = f(X) + Z_H(X) \cdot R(X)$$

H 上では $\tilde{f} = f$ である一方、 $L \setminus H$ 上の評価値は R によってランダム化される。Ligero はこれを randomized RS encoding と blind codeword として実装する。Aurora は bounded

independence を明示的に利用し、sumcheck には random self-reduction を加える。ZK-STARK は trace polynomial をマスクし、composition oracle の照会が秘密 trace を直接露出しないようにする。

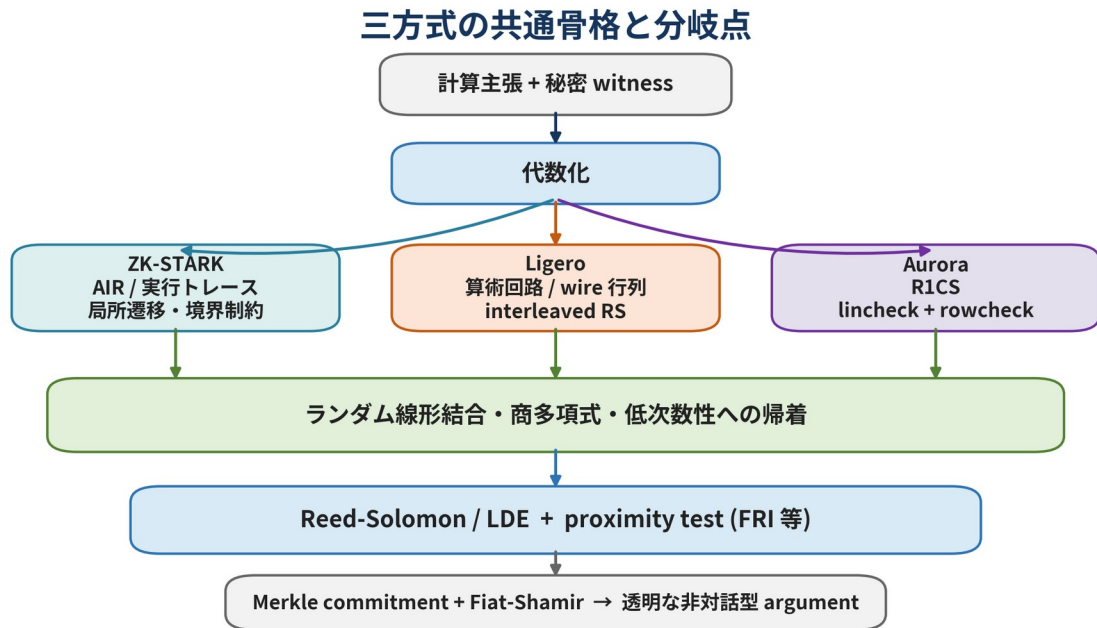


図 2 三方式の共通骨格。差は主として代数化と整合性検査の設計にある。

本稿作成。原理は [1, Sec. 2]、[2, Sec. 4-5]、[3, Sec. 2-9] に基づく。

3. ZK-STARK: 行トレースと低次性

3.1 方式の設計目標と計算モデル

2018 年の ZK-STARK 論文は、透明性、hash-based な耐量子志向、quasi-linear prover、polylogarithmic verifier を同時に実現することを目標とする[1]。中心となる計算モデルは、プログラムを有限状態機械として実行し、各時刻の状態を execution trace に記録する AIR である。明示的な巨大回路を入力として読むのではなく、短い program description と長い time bound を扱うため、uniform computation、特に長い逐次計算に適する。

原論文の形式的な流れは AIR → APR → ALI → 2 個の RS proximity problem → FRI である。APR は、メモリアクセスや状態遷移を代数的に配置・routing する層である。単純な固定幅逐次計算では AIR から直接 composition polynomial を構成できるため、教育的説明や現代的な STARK 実装では APR が前面に出ない場合も多い。しかし、原論文の一般計算への拡張では APR が重要である。

ZK-STARK の証明パイプライン

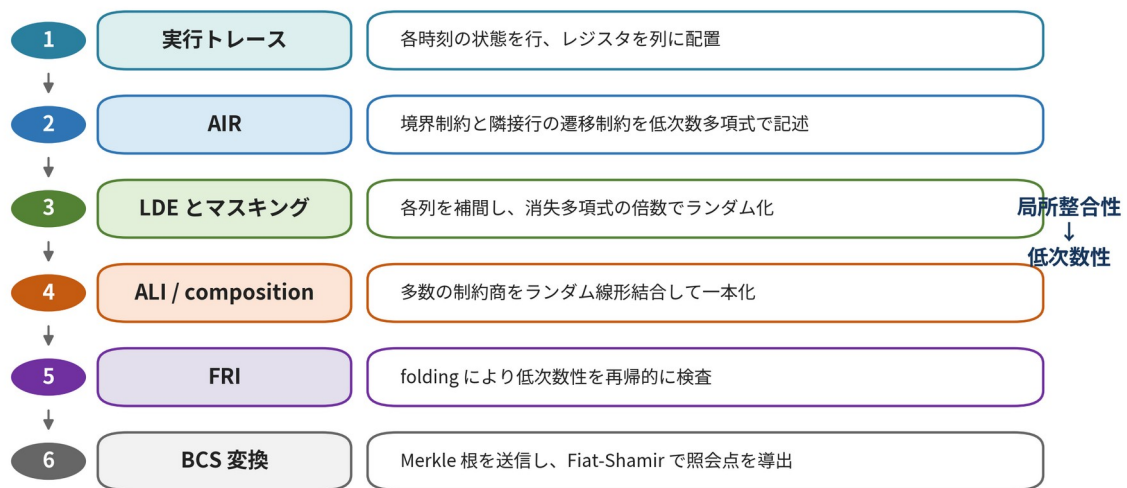


図 3 ZK-STARK の証明パイプライン

[1, Fig. 8, Sec. 2, Appendix B-D] をもとに本稿で再構成。

3.2 AIR: trace、遷移制約、境界制約

幅 w 、長さ T の execution trace を $W \in F^{\wedge\{T \times w\}}$ とする。第 t 行は時刻 t の状態、第 j 列はレジスタ j の時間系列である。各列を domain H 上の関数と見なし、低次数多項式 $f_j(X)$ に補間する。隣接状態の正しさは、現在行 $f_j(X)$ と次行 $f_j(gX)$ または $f_j(X+\delta)$ に対する低次数制約で表現する。ここで g または δ は trace domain 上の next map である。

境界制約は、初期状態、公開入力、最終出力等を固定する。遷移制約だけでは cycle 全体を回転させた不正 trace を排除できないため、境界条件は不可欠である。また最終行には next state が存在しないため、遷移制約を課す点集合から最終点を除く selector または zeroifier を用いる。

3.3 ALI と composition polynomial

各制約 $C_j(X)$ が active domain H_j 上でゼロであることは、 $C_j(X)/Z_{H_j}(X)$ が多項式であることと同値である。証明者はこれらの商を計算し、検証者が選ぶランダム係数 ρ_j で線形結合する。

$$D(X) = \sum_j \rho_j \cdot C_j(X) / Z_{H_j}(X)$$

ランダム係数は、複数制約のうち少なくとも一つが破れている場合に、それらの誤差が偶然相殺する確率をおおむね $1/|F|$ へ抑える。多数の AIR 制約を一本の composition oracle へまとめることが ALI の本質である。原論文では register-wise encoding とランダム線形結合により、以前の arithmetization に存在した大きな degree blow-up を削減している[1, Sec. 2.3]。

3.4 FRI: 低次数性を再帰的に半分へ畳み込む

FRI は Reed-Solomon codeword への近接性を検査する IOP of proximity である。乗法 domain で x と $-x$ を対にすると、多項式 f は偶数部と奇数部を用いて次のように分解できる。

$$f(X) = f_{\text{even}}(X^2) + X \cdot f_{\text{odd}}(X^2)$$

検証者がランダム β を送ると、証明者は半分の domain 上に $f_{\text{next}}(Y) = f_{\text{even}}(Y) + \beta f_{\text{odd}}(Y)$ を commit する。 f が低次数なら f_{next} の次数はほぼ半分になる。これを再帰し、最後に定数または小次数多項式を直接検査する。照会フェーズでは、元 oracle の対 $f(x)$, $f(-x)$ と fold 後の $f_{\text{next}}(x^2)$ の整合性を確認する。

FRI の proof oracle 自体は長いですが、Merkle commitment 後に開示されるのは各 round の少数点と authentication path である。そのため最終 argument size はおおむね $O(\log^2 T)$ となる一方、prover は全 oracle を生成するため quasi-linear な計算と大きな memory bandwidth を必要とする。

3.5 零知識化と非対話化

trace column f_j は、 H 上の計算値を保持したまま $f_j + Z_H R_j$ に置き換えられる。 R_j の次数を query bound 以上に確保すれば、LDE domain 上の少数評価は秘密 trace から統計的に独立となる。composition polynomial も、マスク後の trace と整合するように構成される。

IOP 段階では情報理論的な零知識性と健全性を議論できるが、Merkle 木と Fiat-Shamir を用いた STARK は計算量的 argument である。原論文は interactive STARK と random-oracle model の non-interactive STARK を区別している[1, Sec. 3.3]。

3.6 計算量と設計上の強み・弱み

表 2 ZK-STARK の設計特性

観点	ZK-STARK の性質	設計上の帰結
計算モデル	AIR / program trace	短い program と長い実行時間を扱いやすい。
prover	quasi-linear。一般計算では原論文で $O(T \log^2 T)$ 型	FFT、FRI、Merkle 構築、memory traffic が支配的である。
verifier	program size + $\text{polylog}(T)$ を狙う	逐次計算の再実行より大幅に軽い。
argument size	polylogarithmic。典型的に $O(\log^2 T)$ 型	大規模で漸近優位だが、hash path の定数が大きい。
setup/assumption	透明、hash/RO ベース、耐量子志向	toxic waste はないが、RO モデルと hash security に依存する。
主な弱点	AIR 設計と大規模 oracle 生成	小規模回路では定数項が重く、proof も pairing 系より大きい。

強みは、計算時間 T が巨大であっても program description が短ければ、検証者が T に線形な入力を読む必要がない点にある。弱みは、trace 全体の LDE と FRI oracle を生成・commit する prover 側のメモリ、帯域、FFT コストである。また、どの状態変数と制約多項式を選ぶかが性能を左右するため、arithmetization engineering の比重が大きい。

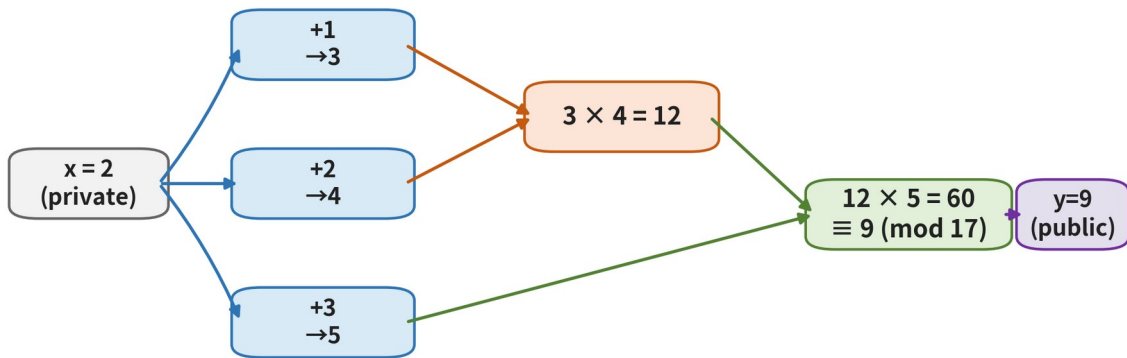
3.7 代表的応用

原論文は、秘匿 DNA データベースに対する検索結果の正しさを公開検証する DPM を proof-of-concept として実装した。これは、秘密データに対する長い逐次処理を、データを開示せずに監査する用途である。また、短い program が長い ledger state transition を実行する状況では、各ノードが全計算を再実行せずに proof だけを検証する scalability layer へ自然に適用できる[1, Sec. 1.2]。

一般化すると、長い VM execution、ロールアップ型の状態遷移、秘匿データベース監査、科学計算の integrity、ログ処理、逐次アルゴリズムの証明に適する。ただし、実際の適性は AIR width、constraint degree、hash function、field、memory layout に依存する。

3.8 F_17 toy computation による STARK 実行例

F_17 上の toy relation: $y=(x+1)(x+2)(x+3)$, $x=2$, $y=9$



同一の関係を、STARK は実行トレース、Ligero は wire 行列、Aurora は R1CS として表現する。

図 4 共通 toy relation の算術回路

本稿作成。F_17 では $60 \equiv 9$ である。

toy 例の安全性

F_17 ではランダム線形結合の失敗確率が最大 $1/17$ 程度であり、暗号的 soundness を満たさない。また、domain と mask degree の余裕も極めて小さい。以下は AIR、商多項式、FRI folding の構造を数値で追うための例である。

公開主張を $y=9$ とし、秘密 witness を $x=2$ とする。逐次計算用に、レジスタ x 、factor index i 、accumulator a を用いる。初期状態は $i=1$, $a=1$ であり、遷移は $x'=x$, $i'=i+1$, $a'=a(x+i)$ である。三回の遷移後に $a=9$ となる。trace domain には F_{17}^* の位数 4 部分群 $H=\{1,4,16,13\}$ を用い、 $\text{next}(h)=4h$ とする。

表 3 toy STARK の execution trace

h	x	i	a	意味
1	2	1	1	初期状態
4	2	2	3	$1 \cdot (2+1)=3$
16	2	3	12	$3 \cdot (2+2)=12$
13	2	4	9	$12 \cdot (2+3)=60 \equiv 9$

H 上の値を補間すると、以下の多項式を得る。係数はすべて mod 17 である。

$$X(T)=2$$

$$I(T)=11+10T+8T^2+6T^3$$

$$A(T)=2+16T+13T^2+4T^3$$

最終点 13 では遷移を課さないため、active domain を $H_{\text{active}}=\{1,4,16\}$ とする。その消失多項式は次である。

$$Z_{\text{active}}(T)=(T-1)(T-4)(T-16)=T^3+13T^2+16T+4$$

index 遷移と accumulator 遷移の制約多項式を計算すると、完全に Z_{active} で割り切れる。

$$C_i(T)=I(4T)-I(T)-1 = 4 \cdot Z_{\text{active}}(T)$$

$$C_a(T)=A(4T)-A(T)(X(T)+I(T)) = Z_{\text{active}}(T)(11+15T^2+10T^3)$$

検証者のランダム係数を例として $\rho_i=5$ 、 $\rho_a=7$ と固定すると、商の composition polynomial は次のようになる。実プロトコルでは、これらの係数は trace commitments 後に生成される。

$$D(T)=5 \cdot 4 + 7(11+15T^2+10T^3)=12+3T^2+2T^3$$

表 4 位数 8 domain $D=\{1,2,4,8,16,15,13,9\}$ 上の composition evaluations

T	1	2	4	8	16	15	13	9
D(T)	0	6	1	4	13	8	0	13

FRI round 1 では $D(T)=D_{\text{even}}(T^2)+T D_{\text{odd}}(T^2)$ と分解する。 $D_{\text{even}}(Y)=12+3Y$ 、 $D_{\text{odd}}(Y)=2Y$ である。 $\beta=6$ なら $D_1(Y)=12+15Y$ となる。さらに $\beta=5$ で fold すると定数 $D_2=2$ となる。

照会整合性を一点だけ確認する。 $x=2$ と $-x=15$ の値は $D(2)=6$ 、 $D(15)=8$ である。2 の逆元等を用いると、 $Y=4$ における even 値は 7、odd 値は 8 であり、 $7+6 \cdot 8=55 \equiv 4$ となる。これは $D_1(4)=4$ と一致する。

Toy STARK: AIR trace から FRI folding まで

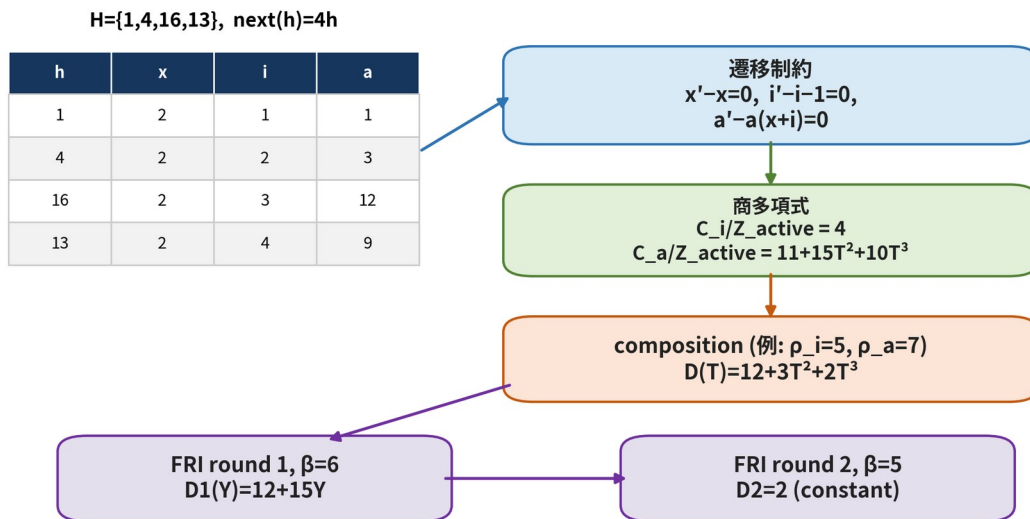


図 5 toy STARK の trace、商多項式、FRI folding

本稿の計算。すべて F_{17} 上。

零知識化では、例えば $A(T)$ を $A(T) + Z_H(T)(5+T)$ に置き換える。H 上の trace 値は不変であるが、LDE domain 上の評価はマスクされる。証明者は trace LDE、composition oracle、各 FRI round を Merkle commit し、Fiat-Shamir から得た query 位置について必要な値と path を開示する。検証者は境界条件、composition identity、FRI folding、最終低次数を確認して受理する。

4. Liger: interleaved RS と平方根通信量

4.1 設計思想: MPC から ZKIPCP へ

Liger は、MPC protocol の内部実行を証明者が「頭の中で」再現し、その一部 view をランダムに開示する MPC-to-ZK の系譜から生まれた方式である。2022 年の拡張版は、CCS 2017 版に完全証明、tightened soundness analysis、Fiat-Shamir 解析、改良実装を追加している[2]。

直接構成では、回路の wire values を少し冗長な extended witness としてほぼ平方の $m \times l$ 行列へ配置し、各行を Reed-Solomon 符号化する。検証者は、行のランダム線形結合を受け取り、少数の列だけを照会する。行数 m と列数 l をともに約 $\sqrt{s}$ に選ぶことで、長い線形結合 codeword の送信量と列開示量を均衡させ、通信量を $O(\sqrt{s})$ にする。

Liger: witness 行列と三つの検査

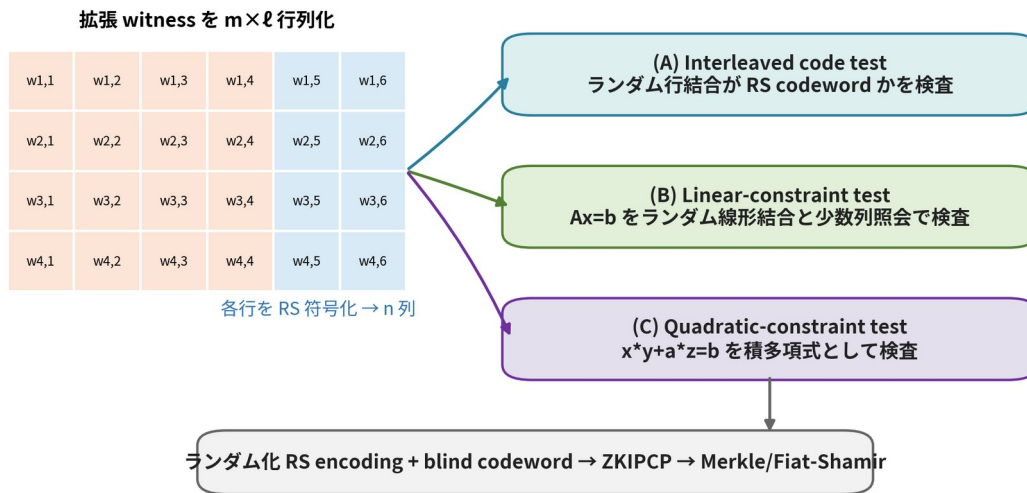


図 6 Liger の witness 行列と三つの検査

[2, Sec. 4] をもとに本稿で再構成。

4.2 Interleaved Reed-Solomon code test

m 本の RS codeword を行に持つ行列 $U \in F^{\wedge\{m \times n\}}$ を interleaved codeword とみなす。検証者は $r \in F^{\wedge m}$ をランダムに選び、証明者に $w = r^{\wedge T} U$ を返させる。w が RS codeword であることを検査し、さらにランダムに選んだ t 列 Q について $w_j = \sum_i r_i U_{\{i,j\}}$ を確認する。

不正行列が正しい interleaved code から e 列以上離れている場合、ランダム行結合が偶然近い codeword になる確率と、 t 列照会がすべて誤り位置を外す確率を合成する。改良解析では RS 符号

について $e < d/2$ の範囲で、おおむね $n/|F| + (1-e/n)^t$ 型の境界が得られる[2, Sec. 4.1.1, Appendix C]。

4.3 Linear-constraint test

message x が $Ax=b$ を満たすことは、ランダム r に対して $r^T Ax = r^T b$ が成立することで検査できる。不正な x に対してこの等式が成立する確率は $1/|F|$ である。Ligero は、 $r^T A$ の係数を各 message block 上の低次数多項式へ補間し、row polynomial との積を合計した $q(X)$ を証明者に送らせる。検証者は、message points での sum が $r^T b$ に一致することと、ランダム列で q の評価が U と一致することを確認する。

$$\Pr[\text{false linear relation passes}] \leq 1/|F| + ((e+k+1)/n)^t$$

第一項はランダム線形結合の偶然の消失、第二項は不正な q と正しい q が一致する有限個の点および error columns だけを query が選ぶ確率に対応する。

4.4 Quadratic-constraint test

算術回路の乗算ゲートは、vector 形式で $x \circ y + a \circ z = b$ と表せる。各 row の補間多項式 p_x 、 p_y 、 p_z について $p_x p_y + p_a p_z - p_b$ を形成すると、次数はおおむね $2k-2$ となる。検証者は m 個の row constraints をランダム係数で集約した $p_0(X)$ を受け取り、message points でゼロであることと、ランダム列で元 oracles と整合することを確認する。

$$\Pr[\text{false quadratic relation passes}] \leq 1/|F| + ((e+2k)/n)^t$$

4.5 零知識化

第一の漏洩源は、検証者が codeword の少数列を直接読むことである。これには、message points 上の値を固定しつつ余分な degree freedom を一様を選ぶ randomized RS encoding を用いる。第二の漏洩源は、検証者が $r^T U$ という線形結合を丸ごと受け取ることである。これには、所定 message を 0 に decode する random codeword u_0 を加え、 $r^T U + u_0$ を開示する。

linear test と quadratic test にも、それぞれ sum-zero message または zero message を持つ blind codeword を加える。ZKIPCP 段階では honest-verifier perfect zero knowledge を達成し、Merkle commitment と BCS 型変換を介して random-oracle model の非対話型 argument へ移行する[2, Sec. 4.6-5.2]。

4.6 計算量、強み、弱み

表 5 Ligero の設計特性

観点	Ligero の性質	設計上の帰結
計算モデル	算術/Boolean circuit、extended witness matrix	回路トポロジーより gate 数に依存しやすい。
communication	$O(\sqrt{s})$	小中規模では定数が小さく、巨大規模では polylog 方式に劣る。

観点	Ligero の性質	設計上の帰結
prover	RS encoding 中心の $O(s \log s)$ 型	対称暗号と FFT 中心で並列化しやすい。
verifier	基本的に回路規模へ線形の準備・演算	strictly succinct verifier ではない。
round	IPCP として少数 round。非対話化可能	設計が比較的単純である。
amortization	同一回路の複数 instance に有利	batch proof で通信・検証の平均コストを下げられる。

Ligero の主要な強みは、複雑な routing network を用いず、RS encoding、ランダム線形結合、少数列照会という単純な機構で透明な sublinear argument を構成できる点である。改良版では小から中規模回路に対する proof size と prover time が強い。一方、 $O(\sqrt{s})$ 通信量は漸近的には Aurora や STARK の polylog proof に追い越される。

4.7 代表的応用

同一回路を多数 instance へ適用する batch 検証、暗号 primitive の preimage、回路トポロジーが不規則な算術回路、proof 生成側の実装単純性を重視する outsourced computation に適する。原論文は SHA-256 preimage に対して 2^{-40} soundness で約 35KB という例を報告し、multi-instance setting で amortized communication と verification を改善できるとする[2, Abstract, Sec. 5.4-6]。

4.8 F₁₇ toy computation による Ligero 実行例

wire vector を $w=(1,x,x+1,x+2,x+3,(x+1)(x+2),y,0)=(1,2,3,4,5,12,9,0)$ とし、 2×4 の message matrix へ配置する。

$$W = [[1,2,3,4],[5,12,9,0]]$$

message points を $\zeta=\{0,1,2,3\}$ 、codeword evaluation points を $\eta=\{4,5,6,7,8,9,10,11\}$ とする。 $Z_\zeta(X)=X(X-1)(X-2)(X-3)$ である。各 row の一意補間多項式は $p_1=1+X$ 、 $p_2=5+2X+10X^2+12X^3$ である。零知識化のため、以下のランダム化を例として選ぶ。

$$p_{1_tilde} = p_1 + Z_\zeta(4+2X) = 1+11X+15X^2+15X^3+9X^4+2X^5$$

$$p_{2_tilde} = p_2 + Z_\zeta(7+5X) = 5+11X+6X^2+8X^3+11X^4+5X^5$$

表 6 toy Ligero の randomized RS codewords

η	4	5	6	7	8	9	10	11
row 1	4	3	4	15	0	0	16	11
row 2	8	12	3	6	0	8	0	0

interleaved code test で検証者が $r=(3,5)$ を選び、blind codeword として $u_0=Z_\zeta(6+3X)$ を用いると、証明者が返す codeword $v=3 \cdot \text{row1}+5 \cdot \text{row2}+u_0$ は次になる。

$$v=(8,5,14,9,12,8,13,6)$$

$\eta=7$ の列を query すると、 $\text{row1}=15$ 、 $\text{row2}=6$ 、 $u_0(7)=2$ であり、 $3 \cdot 15+5 \cdot 6+2=77 \equiv 9=v(7)$ である。検証者は同様の照会を t 列で行い、 v が次数 6 未満の RS codeword であることも確認する。

Toy Ligero: randomized RS 行列と一列照会

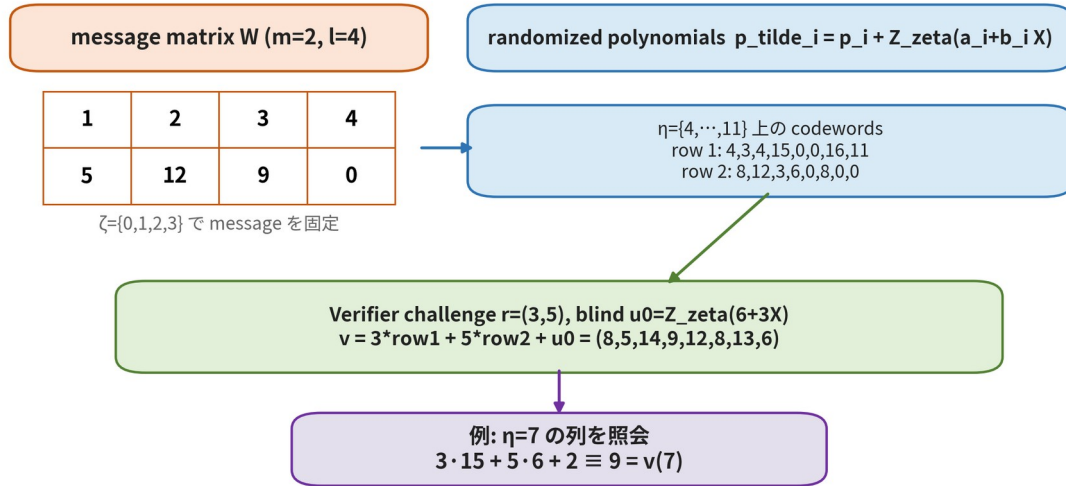


図7 toy Ligero の randomized encoding と列照会

本稿の計算。すべて F_{17} 上。

linear constraints は、 $c_1=w_2-w_1-w_0$ 、 $c_2=w_3-w_1-2w_0$ 、 $c_3=w_4-w_1-3w_0$ 、 $c_4=w_6-9w_0$ である。検証者が係数 $(2,7,4,3)$ で集約すると、 w に対する係数 vector は $(13,4,2,7,4,0,3,0)$ となり、内積は $102 \equiv 0$ である。Ligero の linear test は、この集約を message-domain の sum と random columns の整合性として検査する。

quadratic constraints は $q_1=w_2w_3-w_5=3 \cdot 4-12=0$ 、 $q_2=w_5w_4-w_6=12 \cdot 5-9=51 \equiv 0$ である。係数 $(6,11)$ による集約も 0 である。実際の protocol では、これらの積を補間多項式上で構成した $p_0(X)$ を送信し、message points でゼロ、query columns で元 codewords と一致することを確認する。

最後に、全 codeword columns と prover messages を Merkle commit し、Fiat-Shamir で r 、linear/quadratic aggregation coefficients、query set Q を導出する。proof は開示された少数列、集約 codewords またはその記述、Merkle paths から構成される。

5. Aurora: R1CS、lincheck、単変数 sumcheck

5.1 設計目標と R1CS モデル

Aurora は、明示的な R1CS instance に対する透明な zkSNARK を構成する。R1CS は、public input v と private witness w を連結した $z=(1,v,w)$ に対し、三つの疎行列 A 、 B 、 C が $Az \circ Bz = Cz$ を満たすかを問う。算術回路より表現力が高く、field arithmetic を自然に扱える一方、constraint system 自体は明示的である。

Aurora の IOP は、oracle proof length $O(N)$ 、query complexity $O(\log N)$ 、prover $O(N \log N)$ 、verifier $O(N)$ を達成する。BCS 変換後の argument size は $O_\lambda(\log^2 N)$ である[3, Theorem 1.1-1.2]。verifier が $O(N)$ なのは弱点ではなく、明示的な R1CS を入力として受け取る以上、その疎行列記述を読むだけで $\Omega(N)$ が必要だからである。

Aurora: R1CS を rowcheck と lincheck に分解

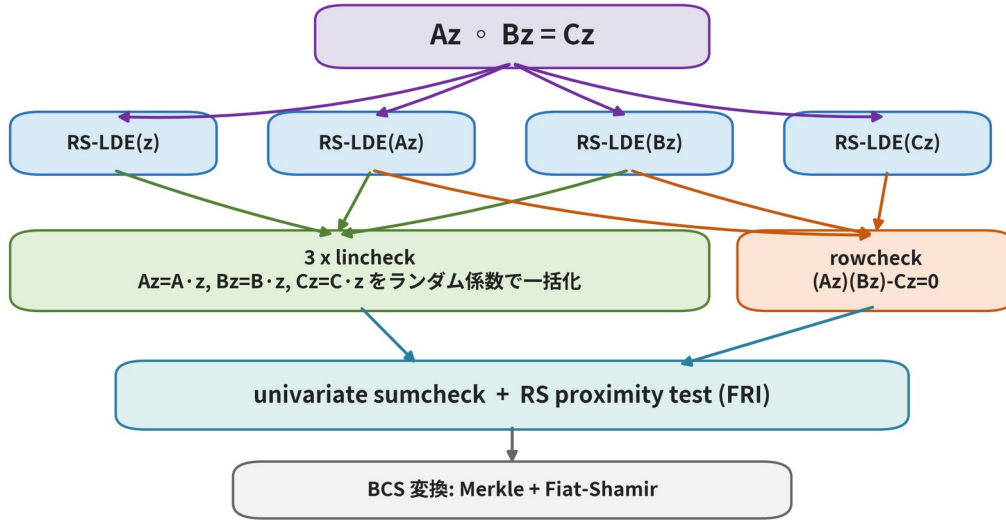


図8 Aurora の R1CS 分解

[3, Fig. 3, Sec. 2, 5-9] をもとに本稿で再構成。

5.2 R1CS を三つの lincheck と一つの rowcheck へ分解する

証明者は z 、 Az 、 Bz 、 Cz の低次数拡張を oracle として送る。検証者は三つの lincheck で $y_A = Az$ 、 $y_B = Bz$ 、 $y_C = Cz$ を確認し、rowcheck で $y_A \circ y_B = y_C$ を確認する。これにより、R1CS の疎な線形演算と pointwise multiplication を別々の subprotocol へ分離できる。

rowcheck は比較的直接的である。多項式 f_A, f_B, f_C が constraint domain H_1 上でゼロなら、 $\mathbb{Z}_{\{H_1\}}$ で割った商が低次数である。したがって、商を明示的に送らず、virtual oracle として ratio の Reed-Solomon rate を検査できる。

$$q_{\text{row}}(X) = (f_A(X)f_B(X) - f_C(X)) / \mathbb{Z}_{\{H_1\}}(X)$$

5.3 lincheck を sumcheck へ帰着する

linear relation $x=My$ が偽であれば、残差 $e=x-My$ は非ゼロである。検証者は単一の $\alpha \in F$ を選び、power vector $r(\alpha)=(1,\alpha,\dots,\alpha^{m-1})$ との内積 $h(\alpha)=\langle r(\alpha),e \rangle$ を検査する。 h は次数 $m-1$ 以下の非ゼロ多項式であるため、Schwartz-Zippel 型の議論により $h(\alpha)=0$ となる確率は高々 $m/|F|$ である。

この内積は、constraint domain 上の二つの sum へ展開できる。Aurora は両 sum を一つの affine/multiplicative coset 上の univariate sumcheck へまとめる。三つの lincheck は同じ domain を用いるため、ランダム線形結合により一回の amortized sumcheck へ束ねられる。

5.4 単変数 sumcheck

多変数 sumcheck は tensor structure を用いて変数を一つずつ減らすが、Reed-Solomon codeword は単変数であり同じ再帰が使えない。Aurora の主要貢献は、additive または multiplicative coset H 上の sum claim を、商と低次数性へ帰着する単変数 sumcheck である。

乗法部分群 H 、 $\deg f < d$ の場合、 f を remainder と Z_H の倍数へ分解する。 $\deg \text{remainder} < |H|$ であり、 H 上の非定数 monomial の総和が 0 になるため、sum は remainder の定数項だけで決まる。一般に $\deg f > |H|$ であっても、 $f=g+Z_H h$ として H 上の値を保つ低次数 remainder g を取り出せる。証明者は h の oracle を送り、検証者は $f-Z_H h$ が必要な低次数 subcode に入ること proximity test で確認する。

$$\sum_{a \in H} f(a) = |H| \cdot g(0) \quad (\text{multiplicative-coset case, } \deg g < |H|)$$

additive subspace の場合は、sum が $X^{|H|-1}$ 係数で決まるという Byott-Chapman 型の恒等式を用いる。いずれも「sum claim を特殊な Reed-Solomon subcode への membership へ変換する」点が共通する。

5.5 RS-encoded IOP から通常の IOP へ

rowcheck、lincheck、sumcheck を記述した段階では、証明者 oracles が所定 rate の RS codeword であることを仮定した RS-encoded IOP である。Aurora は generic compiler により、この仮定を low-degree test へ置き換える。複数 oracles と rational constraints をランダム線形結合し、一つまたは少数の FRI instances として検査する。

これにより、core protocol は R1CS の代数に集中し、codeword validity は FRI へ委譲される。libiop はこの分離を software abstraction として実装し、IOP subcomponents、BCS transform、field/FFT/Merkle infrastructure を組み合わせられる設計である[3, Sec. 10]。

5.6 零知識化

z 、 Az 、 Bz 、 Cz を一意な LDE で送ると、少数照会から秘密 witness の線形情報が漏れる。Aurora は、message domain 上の値を固定したまま degree を b だけ増やした random

polynomial を選び、evaluation domain を message domain と disjoint にする。これにより、最大 b 個の distinct queries は一様な field elements として見える。

しかし、sumcheck で Z_H による division を行くと mask が消える場合がある。そこで、証明者は random RS codeword q とその sum β を先に送る。検証者 challenge c に対して、元 claim f の代わりに $c \cdot f + q$ の sum claim を証明する。 q が一様であるため、sumcheck の入力自体がランダム化され、soundness は false claim が偶然 target sum へ一致する $1/|F|$ で保たれる [3, Sec. 2.3, 5.1, 7.1]。

5.7 計算量、強み、弱み

表 7 Aurora の設計特性

観点	Aurora の性質	設計上の帰結
計算モデル	明示的 R1CS	既存 circuit compiler と親和性が高い。
IOP proof/query	$O(N)$ / $O(\log N)$	Merkle 化後の argument は $O(\log^2 N)$ 。
prover	$O(N \log N)$	routing network を避け、STARK の R1CS 変換より軽い。
verifier	$O(N)$	R1CS matrix を読む必要があり、uniform program には不利。
中核技術	univariate sumcheck	lincheck を RS code 上で効率化する。
主な弱点	明示 R1CS に限定、field/FFT/FRI 定数	長い VM trace を R1CS 化すると instance が膨らむ。

Aurora の強みは、explicit non-uniform computation を R1CS として与えた場合に、Ligero の $\sqrt{N}$ query を $\log N$ へ減らし、STARK が routing/memory checking に支払う overhead を回避する点である。弱みは、program が短く実行時間だけが長い uniform computation であっても、展開済み R1CS を入力として扱うため、verifier が instance size に線形となる点である。

5.8 代表的応用

既存の R1CS compiler から生成される算術回路、暗号 protocol、資産保有証明、秘密入力を含む business logic、明示的 constraint system の監査に適する。特に、trusted setup を避けつつ R1CS ecosystem を活用したい場合に自然な選択である。複数 instance が同じ A、B、C を共有する場合には、lincheck/sumcheck の amortization が有効である。

5.9 F₁₇ toy computation による Aurora 実行例

同じ関係を、補助変数 $t=(x+1)(x+2)$ だけを導入した 2 制約 R1CS として表す。public $y=9$ 、private witness は $x=2$ 、 $t=12$ であり、 $z=(1,y,x,t)=(1,9,2,12)$ とする。

表 8 toy relation の R1CS

row	A row	B row	C row	constraint
1	(1,0,1,0)	(2,0,1,0)	(0,0,0,1)	$(x+1)(x+2)=t$
2	(0,0,0,1)	(3,0,1,0)	(0,1,0,0)	$t(x+3)=y$

$Az=(3,12)$ 、 $Bz=(4,5)$ 、 $Cz=(12,9)$ であり、 $3 \cdot 4=12$ 、 $12 \cdot 5=60 \equiv 9$ である。4点 domain へ padding し、 $H=\{1,4,16,13\}$ 上に $z=(1,9,2,12)$ 、 $Az=(3,12,0,0)$ 、 $Bz=(4,5,0,0)$ 、 $Cz=(12,9,0,0)$ を配置する。evaluation domain は H と disjoint な $L=3 \cdot \{1,2,4,8,16,15,13,9\}=\{3,6,12,7,14,11,5,10\}$ とする。

$Z_H=X^4-1$ を使い、各補間多項式へ一次 mask を加えた例は次である。これらは H 上で所望の値を持ち、 $\text{degree} < 6$ である。

$$f_z=4+4X+4X^2+X^3+2X^4+3X^5$$

$$f_A=4+9X+2X^2+4X^4+X^5$$

$$f_B=8+8X+4X^2+6X^3+7X^4+5X^5$$

$$f_C=12+9X+5X^2+12X^3+6X^4+2X^5$$

表 9 toy Aurora の masked RS codewords

L point	3	6	12	7	14	11	5	10
f_z	1	9	11	0	12	14	11	8
f_A	4	0	13	5	8	14	12	10
f_B	6	12	2	4	9	8	5	1
f_C	3	3	2	11	12	4	3	7

rowcheck では $f_A f_B - f_C$ が H 上でゼロであるため Z_H で割り切れる。実際、商は次である。

$$q_{\text{row}}=14+7X+3X^2+4X^3+10X^5+5X^6$$

lincheck の第一行列 A について、 $\alpha=6$ を選ぶ。constraint-side power vector は $r=(1,6,6^2,6^3)=(1,6,2,12)$ である。padding 後の A に対して $r^T A=(1,0,1,6)$ となる。したがって $r^T(Az)=3+6 \cdot 12=75 \equiv 7$ 、 $(r^T A)z=1+2+6 \cdot 12=75 \equiv 7$ である。

これを domain 上の sum へ展開する。 p_1 の H 上の値を r 、 p_2 の H 上の値を $r^T A$ とすると、 $q(a)=p_1(a)(Az)(a)-p_2(a)z(a)$ は $(2,4,15,13)$ となり、総和は $34 \equiv 0$ である。補間多項式は $q(X)=10X+9X^3$ である。 H は位数 4 の乗法部分群なので、 $\text{degree} < 4$ の q について $\sum_H q=4q(0)=0$ である。Aurora の univariate sumcheck は、この sum claim と q の低次数性を検査する。

Toy Aurora: 2 制約 R1CS と sumcheck

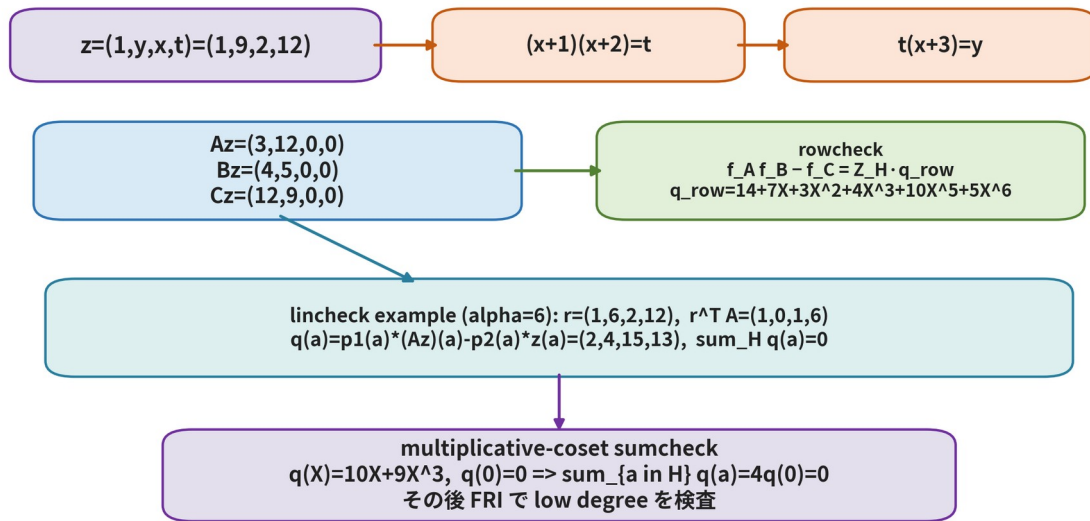


図9 toy Aurora の R1CS、rowcheck、lincheck、sumcheck

本稿の計算。すべて F_{17} 上。

三つの lincheck をランダム線形結合し、一回の sumcheck へ amortize する。rowcheck quotient、sumcheck proof oracle、masked witness oracles を generic compiler で FRI へ接続し、Merkle commitment と Fiat-Shamir で非対話化する。検証者は数点の codeword values、rational constraints、FRI folding consistency を確認する。

6. 三方式の横断比較

6.1 何が本質的に異なるか

表 10 三方式のアーキテクチャ比較

軸	ZK-STARK	Ligero	Aurora
計算表現	AIR / execution trace	extended witness matrix	R1CS
整合性の核	遷移・境界の商多項式	interleaved code + linear/quadratic tests	3 linchecks + rowcheck
主要圧縮技術	ALI composition + FRI	random row combinations + column sampling	univariate sumcheck + FRI
最終 argument	Merkle + Fiat-Shamir	Merkle + Fiat-Shamir	Merkle + Fiat-Shamir
得意 workload	長い uniform/sequential computation	中規模回路、batch、単純 prover	明示 R1CS、non-uniform circuit
主なボトルネック	trace LDE、FRI、memory bandwidth	$\sqrt{N}$ communication、column openings	R1CS 読込 $O(N)$ 、FFT/FRI 定数

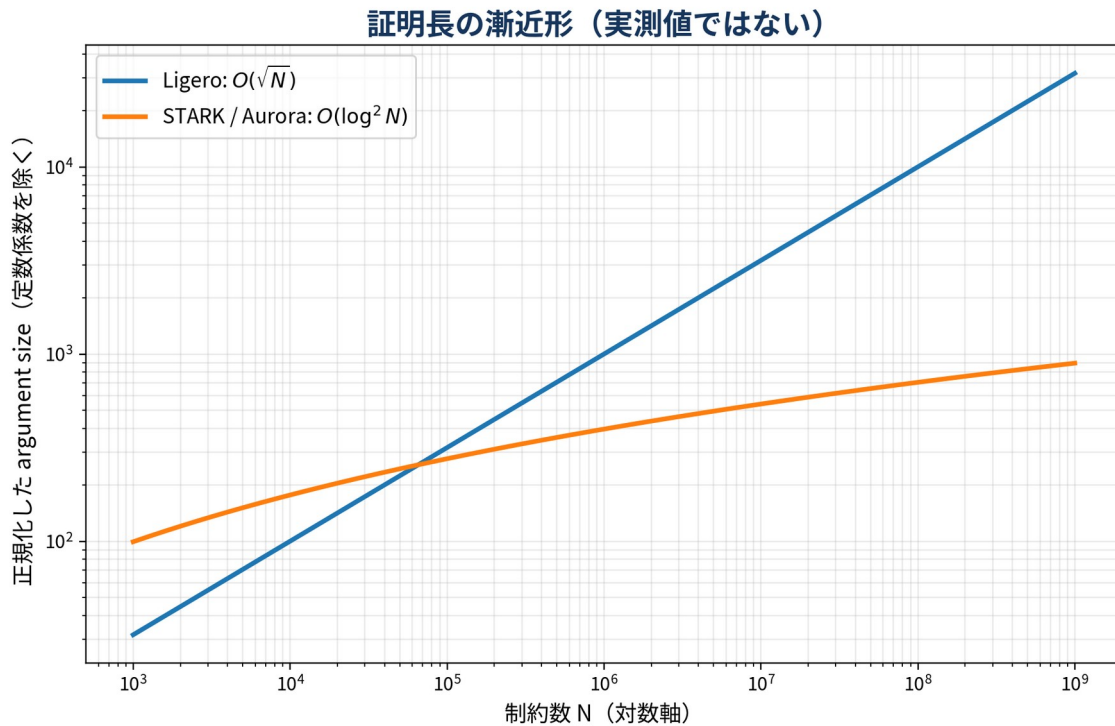
最も重要な差は、検証者が「計算の記述」をどの大きさに受け取るかである。ZK-STARK は短い program と time bound から長い trace を証明できるため、verifier は実行時間 T に線形な記述を読む必要がない。Aurora と Ligero は明示回路または R1CS を入力とするため、verifier がその記述を読むコストは避けられない。

6.2 漸近計算量

表 11 N-gate explicit arithmetic circuit へ適用した場合の情報理論的 protocol 比較

方式	protocol type	rounds	oracle proof length	query complexity	prover	verifier
Ligero	IPCP	2	$O(N)$	$O(\sqrt{N})$	$O(N \log N)$	$O(N)$
ZK-STARK	IOP	$O(\log N)$	$O(N \log N)$	$O(\log N)$	$O(N \log^2 N)$	$O(N)^*$
Aurora	IOP	$O(\log N)$	$O(N)$	$O(\log N)$	$O(N \log N)$	$O(N)$

出典: [3, Fig. 1]。*STARK を明示 N-gate circuit へ変換した比較である。短い program で表せる uniform computation では STARK verifier は program size + $\text{polylog}(T)$ となり得る。



注: 実際の交点は field、security parameter、Merkle 認証路、実装定数に強く依存する。

図 10 最終 argument size の漸近形。定数項を除いた概念図であり実測値ではない。

Ligerio $O(\sqrt{N})$ 、STARK/Aurora $O(\log^2 N)$ は [2, Theorem 1.1]、[3, Fig. 2] に基づく。

Ligerio は round 数が少なく構造も単純であるが、query complexity が $\sqrt{N}$ である。Aurora は round を $\log N$ へ増やす代わりに query を $\log N$ へ減らす。STARK は program execution を routing するため oracle proof と prover に追加 $\log$ 因子を支払うが、uniform computation に対する verifier succinctness を得る。

6.3 零知識機構の比較

表 12 零知識化の設計比較

方式	local query の隠蔽	線形結合・派生 oracle の隠蔽	証明上の注意
ZK-STARK	trace polynomial に $Z_H R$ を加える	masked trace から composition を生成	boundary と mask degree の整合が重要
Ligerio	randomized RS encoding	zero/sum-zero blind codeword を加える	affine test の soundness 解析が必要
Aurora	randomized LDE、bounded independence	random self-reduction $c f+q$	division で mask が消える問題を回避

6.4 エビデンスに基づく具体比較

実測値の読み方

以下の値は「方式そのものの恒久的順位」ではない。2018-2022 年の特定実装、field、security level、

workload、hardware に依存する。特に 2022 年 Ligerio 論文は、他方式との比較が異なる machine と field に基づくため not apples-to-apples であると明記する。

表 13 歴史的に報告された proof length の例

source/condition	Ligerio	ZK-STARK	Aurora	解釈
Aurora 2019, $N=10^6$, 128-bit, F_2^{192}	約 4.0 MB	約 3.2 MB	約 130 kB	同一 libiop 系比較では Aurora が最小
Ligerio 2022, $N=2^{20}$, 128-bit table	602 kB	—	1,603 kB	改良 Ligerio が小さいが field/instance 差あり
Ligerio 2022, $N=2^{15}$	135 kB	—	931 kB	小中規模で改良定数が効く

1 行目: [3, Fig. 2]。2-3 行目: [2, Table 2]。後者は Ligerio 側が比較条件の不一致を明記している。

2019 年 Aurora 論文の結論は、同じ field と security level で R1CS を比較した当時の libiop 実装において Aurora proof が Ligerio と STARK より一桁以上短いというものである。2022 年 Ligerio 改訂版は soundness analysis と parameters を更新し、別条件の表では小中規模で Aurora より短い proof を報告する。この差は、漸近量だけで実用性能を推定できないことを示す。

表 14 ZK-STARK 原論文の DPM 測定例

項目	2018 ZK-STARK DPM proof-of-concept
security	60-bit
最大生成 instance	$n=2^{20}$ DNA profiles
秘密 dataset size	約 43 MB
time compression rate	約 1/6
witness-size compression rate	約 1/100
compression threshold	time: 2.8×10^5 profiles, size: 9×10^3 profiles

出典: [1, Sec. 1.1, Fig. 1, Appendix A]。特定 hardware と逐次 DPM workload に対する 2018 年の測定である。

DPM 測定は STARK の優位が「大きな逐次計算で現れる」ことを示す一方、小規模・低 depth・反復回路では他方式が有利になり得ると原論文自身が述べる。したがって、benchmark は target workload の width、depth、uniformity、public statement size を揃えて設計すべきである。

6.5 強みと弱みの総括

表 15 方式選択の要約

方式	強み	弱み	最も自然な採用条件
ZK-STARK	uniform computation、polylog verifier、巨大 trace での漸近性	proof 定数、メモリ、AIR 設計	短い program で長い逐次計算を証明する
Ligerio	単純、batch に強い、改良版の小中規模性能	$\sqrt{N}$ 通信、verifier 非 succinct	中規模回路・多数 instance・prover 簡潔性重視

方式	強み	弱み	最も自然な採用条件
Aurora	R1CS native、polylog proof、routing 不要	verifier $O(N)$ 、明示 constraint 依存	既存 R1CS compiler を透明 proof へ接続する

方式選択の設計空間

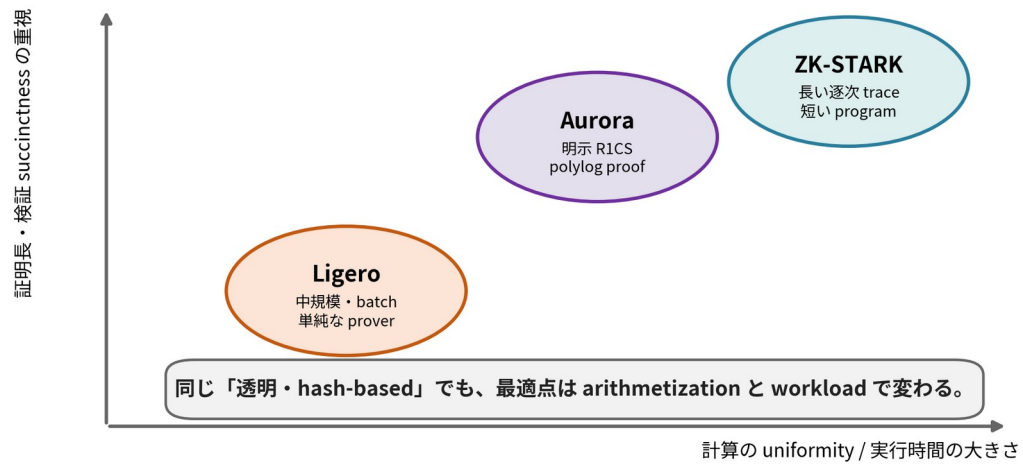


図 11 方式選択の設計空間

本稿の総合評価。

7. 応用設計と方式選択

7.1 分散台帳と状態遷移の validity proof

長い VM execution を短い program description から証明する場合、AIR を直接設計できる ZK-STARK が自然である。全 state transition を R1CS へ展開済みであり、既存 compiler を利用した場合は Aurora が候補となる。Ligero は個々の batch が中規模で、prover 実装の単純性や複数 instance の amortization を重視する場合に適する。

設計時には proof size だけでなく、prover memory、parallel FFT、hash throughput、recursion/aggregation の必要性、on-chain verifier の field compatibility を評価する必要がある。三原論文は recursion を主題としないため、再帰証明を要するシステムでは追加 protocol のコストを別途見積もるべきである。

7.2 秘匿データベース監査

公開 commitment に対応する秘密データを正しく処理したことを証明する用途である。データを逐次走査し、同じ局所遷移を反復するなら STARK が適する。データ件数が中規模で同じ query を多数証明するなら Ligero の multi-instance amortization が有利になり得る。監査ロジックが R1CS としてすでに存在するなら Aurora は integration cost を下げる。

7.3 Outsourced computation と cloud verification

worker が計算結果と proof を返し、client が再実行せず検証する。uniform algorithm と長い runtime には STARK、明示的な算術回路には Aurora、moderate circuit と batch には Ligero という分担が基本である。入力・出力 commitment の binding、データ availability、proof generation failure、resource exhaustion は ZKP 外の system issue であり、end-to-end security では別に扱う必要がある。

7.4 機械学習・科学計算

推論や数値計算を有限体へ写像する際には、quantization、range constraint、division、floating-point semantics が arithmetization cost を支配する。長い recurrent/iterative computation は trace 型、固定 network の明示 constraint は R1CS 型が自然である。Ligero は多数の同型 instance を batch 化する場合に候補となる。証明方式の比較前に、元計算をどの field semantics で表すかを固定しなければならない。

7.5 実装・監査チェックリスト

1. 関係の意味論を固定する。integer overflow、field reduction、bit decomposition、public/private input 境界を明示する。
2. arithmetization の規模だけでなく degree、width、sparsity、domain size、padding 率を測る。
3. prover memory peak、sequential I/O、FFT layout、hash throughput を実機で計測する。
4. soundness error を algebraic test、LDT/FRI、Merkle、Fiat-Shamir に分解して合成する。

- 5.zero-knowledge mask degree が最大 distinct queries を上回ること、message domain と query domain の交差条件を確認する。
- 6.comparison benchmark では field、security parameter、hardware、threads、statement model、setup cost を揃える。
- 7.proof parser、domain separation、transcript ordering、Merkle leaf encoding を仕様化し、malleability を防ぐ。

8. 結論

ZK-STARK、Ligero、Aurora は、いずれも Reed-Solomon 符号、ランダム線形結合、局所照会、Merkle commitment、Fiat-Shamir という共通基盤を持つ。しかし、性能を決めるのは外殻ではなく、計算をどの代数的対象へ変換するかである。ZK-STARK は execution trace の locality を活用し、短い program と長い runtime の差を利用する。Ligero は witness を平方行列へ配置し、code/linear/quadratic test の単純さと $\sqrt{N}$ 通信を交換する。Aurora は R1CS の疎線形代数を lincheck へ分離し、単変数 sumcheck によって logarithmic query を得る。

F_17 toy computation は、三方式が同じ関係を異なる中間表現へ写像することを示した。STARK では $x=2$ は trace column に埋め込まれ、遷移商と FRI で検査された。Ligero では wire values が 2×4 行列へ配置され、randomized row codewords とランダム列照会で検査された。Aurora では二つの R1CS rows へ圧縮され、lincheck、rowcheck、univariate sumcheck へ分解された。

実用上の選択は、証明長の漸近式だけでは決まらない。uniformity、statement representation、batchability、field、security level、memory architecture、proof verifier の配置を含む system-level optimization として決定すべきである。透明な ZKP の設計は、単一の万能方式を探す問題ではなく、計算の構造に最も適した代数化と proximity test の組合せを選ぶ問題である。

付録 A. F_17 toy computation の完全データ

A.1 基本演算

表 A1 共通 toy relation の wire values

項目	F_17 での値
x	2
x+1	3
x+2	4
x+3	5
(x+1)(x+2)	12
y=12·5	60 mod 17 = 9

A.2 STARK polynomial data

表 A2 toy STARK の多項式係数

object	coefficient list (constant to high degree)
I(T)	[11,10,8,6]
A(T)	[2,16,13,4]
Z_active(T)	[4,16,13,1]
C_i(T)	[16,13,1,4]
Q_i(T)	[4]
C_a(T)	[10,6,16,2,15,9,10]
Q_a(T)	[11,0,15,10]
D(T)	[12,0,3,2]
D_1(Y)	[12,15]
D_2	[2]

A.3 Ligero polynomial data

表 A3 toy Ligero の多項式係数

object	coefficient list
Z_ζ(X)	[0,11,11,11,1]
p_1_tilde	[1,11,15,15,9,2]
p_2_tilde	[5,11,6,8,11,5]

object	coefficient list
u_0	[0,15,14,14,5,3]
v polynomial	[11,1,4,14,2]

A.4 Aurora polynomial data

表 A4 toy Aurora の多項式係数

object	coefficient list
f_z	[4,4,4,1,2,3]
f_A	[4,9,2,0,4,1]
f_B	[8,8,4,6,7,5]
f_C	[12,9,5,12,6,2]
q_row	[14,7,3,4,0,10,5]
lincheck q	[0,10,0,9]

再現性

付録の係数は、各 domain 上の Lagrange 補間と mod 17 の多項式除算で機械検算した値である。異なる domain ordering や mask polynomial を選べば oracle values は変化するが、制約充足と検証構造は不変である。

付録 B. エビデンス表と比較上の注意

表 B1 本稿の主要主張と一次資料

主張	一次資料
ZK-STARK の AIR/ALI/FRI 構成	[1, Sec. 2, Fig. 8, Appendix B-D]
DPM compression threshold と測定値	[1, Sec. 1.1, Fig. 1, Appendix A]
Ligero の $\sqrt{\cdot}$ circuit-size communication	[2, Abstract, Theorem 1.1]
Ligero の interleaved/linear/quadratic tests	[2, Sec. 4.1-4.4]
Ligero の ZK 化と FS 変換	[2, Sec. 4.6-5.2]
Ligero 改訂版の proof/time tables	[2, Tables 2-3]
Aurora の R1CS IOP 計算量	[3, Theorems 1.1-1.2]
Aurora の univariate sumcheck	[3, Sec. 2.2, Sec. 5]
Aurora の lincheck/rowcheck/R1CS 構成	[3, Sec. 4.8, 6-9]
2019 年の Ligero/STARK/Aurora 比較	[3, Figs. 1-2, Sec. 11.2]

比較表の数値を再利用する際は、同じ名称の方式でも revision、security analysis、field、hash、query repetition が異なることを確認する必要がある。2019 年 Aurora paper の Ligero は当時の libiop 実装であり、2022 年 Ligero paper の Ligero-128 は tightened analysis と改良実装を含む。したがって、両表を直接連結して performance curve を作ることは不適切である。

「post-quantum secure」は厳密な標準モデル証明を意味しない。三論文は主として collision-resistant hashing または random oracle に基づき、既知の量子攻撃に対して plausibly secure という立場を取る。量子 query model での Fiat-Shamir や hash output length の扱いは、採用する security theorem に応じて別途確認すべきである。

参考文献

- [1] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, Michael Riabzev. “Scalable, transparent, and post-quantum secure computational integrity.” IACR ePrint 2018/046, 2018.
- [2] Scott Ames, Carmit Hazay, Yuval Ishai, Muthuramakrishnan Venkitasubramaniam. “Ligero: Lightweight Sublinear Arguments Without a Trusted Setup.” Extended version, IACR ePrint 2022/1608, 2022; extended abstract in CCS 2017.
- [3] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, Nicholas P. Ward. “Aurora: Transparent Succinct Arguments for R1CS.” IACR ePrint, 2019.
- [4] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, Michael Riabzev. “Fast Reed-Solomon Interactive Oracle Proofs of Proximity.” 2017.
- [5] Eli Ben-Sasson, Alessandro Chiesa, Nicholas Spooner. “Interactive Oracle Proofs.” TCC, 2016.
- [6] Carsten Lund, Lance Fortnow, Howard Karloff, Noam Nisan. “Algebraic Methods for Interactive Proof Systems.” JACM, 1992.
- [7] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, Amit Sahai. “Zero-Knowledge from Secure Multiparty Computation.” STOC, 2007.
- [8] Ivan Damgård, Yuval Ishai. “Scalable Secure Multiparty Computation.” CRYPTO, 2006.
- [9] Amos Fiat, Adi Shamir. “How to Prove Yourself: Practical Solutions to Identification and Signature Problems.” CRYPTO, 1986.
- [10] Irving S. Reed, Gustave Solomon. “Polynomial Codes over Certain Finite Fields.” Journal of the Society for Industrial and Applied Mathematics, 1960.
- [11] Joe Kilian. “A Note on Efficient Zero-Knowledge Proofs and Arguments.” STOC, 1992.
- [12] Silvio Micali. “Computationally Sound Proofs.” SIAM Journal on Computing, 2000.